# 14 Task Queues and Background Jobs

Background tasks are work your backend does outside the main HTTP request-response path so user facing APIs stay fast while heavy or slow operations run asynchronously.

⇒ Why do they exist?

⇒ User-facing requests should be quick. Long running work (emails, image processing, biz DB updates) makes API slow and inline.

⇒ Background processing moves that work to separate workers, improving latency, resilience and allows retries and rate limiting.

## Types of Background Tasks

(1) Event driven async jobs: triggered by an action (user signup → send mail,

user uploads file → generate thumbnails → update analytics).

② Batch Jobs: Process large datasets periodically (nightly billing, reports etc).

③ Scheduled / Cron Jobs: Run at fixed times or intervals (backups at 2 AM)

## IV Core building objects

① Producer: your controller / service that decides "do this later" and enqueues a task with payload or metadata.

② Queue / Broker: Durable storage for tasks (Redis, SQS, Kafka). That would decouple producers from workers.

③ Worker: Background process that pulls tasks and executes them, often scaled horizontally.

④ Scheduler: Components that triggers tasks at time (intervals (cron, disk)

Eg: Typical SaaS apps, like sending emails.

① Processing images / Videos

② Generating reports. (Cron Jobs)
(Celery etc)

④ Sending push notification (phones)

## Task Queue

A TQ is a buffer that stores units of work (tasks) to be processed asynchronously. It's a mechanism that allows the producer (API) and consumer (worker) to operate independently.

Without queue, if the worker crashes the task is lost. If the worker is slow, API hangs. The queue solves this by holding the "state" of the work until it's successfully completed.

(6) How does it work?

Lifecycle:

(1) Enqueue: The producer sends a JSON message (task) to the queue

(2) Dequeue & Lock:

① When a worker picks a task, the task is not removed from the queue.

② Instead the queue makes the task invisible to the other workers for a set period of time (30s), this is called visibility timeout.

③ If the worker crashes mid-process, the task is still in queue. Once the 30s expire, the task becomes visible again and other worker pick it up. This makes sure that the task is not lost if god is down.

(3) Process: Worker executes code

(4) Ack: If successful, the worker sends ACK signal to Queue.

(5) Negative Ack: If worker fail (API is down) it can send a Nack or let timeout expire.

## II Handling failures:

: DLQ.

what if a task fails, no need to try infinetly. and waste resources.
so we use DLQ. After 5th failure the queue moves the tasks to a separate, special queue called DLQ.

## IV Types of Tasks

(1) One-off: A single job trigerred by an event meant to run once (even it retried for reliability).

Eg: Welcome emails,

(2) Batch tasks:

Process large set of items, often offline or low priority, usually trigerred manually or on a schedule.

③ Chained tasks :

A series of dependent steps where later steps runs only if earlier steps succeed.

Eg: Order placed → charge payment → Create order record → reserve inventory → Send conf email.

Video uploaded → transcode → generate thumbnail → update DB → notify.

④ Recurring tasks :

Task that run repeatedly based on time., not events.

Eg: Nightly DB backups, hourly cleanup, daily digest, weekly billing.